

Module 20: Doubly Linked List Deletion and Reversal

1. Current Progress Analysis

You are now at **Module 20**.

Before this module, you already learned the important linked-list ideas needed to understand doubly linked-list deletion and reversal.

Previous Topic	Why It Matters in Module 20
Arrays vs linked lists	You know linked lists change links instead of shifting array values.
Singly linked-list nodes	You know a node stores data and at least one link.
Traversal	You know how to move through nodes using a temporary pointer.
Singly linked-list deletion	You know deletion means disconnecting a node and freeing memory.
Singly linked-list reversal	You know reversal means changing links, not just printing backward.
Circular singly linked lists	You know that some linked lists need special stopping rules.
Doubly linked-list foundations	You know each node has <code>prev</code> , <code>data</code> , and <code>next</code> .
Doubly linked-list insertion	You know doubly linked lists must keep both forward and backward links correct.

In Module 18, you learned the basic structure of a doubly linked-list node:

```
struct Node {
    struct Node *prev;
    int data;
    struct Node *next;
};
```

A normal linear doubly linked list looks like this:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

This means:

- The first node has `prev = NULL`.
- The last node has `next = NULL`.
- Every middle node has both a previous node and a next node.

In Module 19, you learned insertion in a doubly linked list.

Insertion means:

Create a new node and connect it into the list.

Module 20 teaches two new operations:

Deletion: remove a node from the list.
Reversal: reverse the direction of the whole list.

The big difference from singly linked lists is that a doubly linked-list node has two links:

`prev + next`

So every operation must preserve both directions.

2. Big Idea of Module 20

Module 20 has two main parts:

1. **Deletion from a doubly linked list**
2. **Reversal of a doubly linked list**

The main beginner rule is:

Deletion removes one node by connecting its neighbors.
Reversal flips every node's `prev` and `next` links.

Part A: Deletion in a Doubly Linked List

3. What Deletion Means

Deletion means removing a node from the linked list.

Example:

```
Before deletion:  
NULL <- 10 <-> 20 <-> 30 <-> 40 -> NULL  
  
Delete 30.  
  
After deletion:  
NULL <- 10 <-> 20 <-> 40 -> NULL
```

The node containing `30` is removed.

But deletion has two responsibilities:

1. **Disconnect the node from the list**
2. **Release the node's heap memory**

In C, if the node was created using `malloc`, we release it using:

```
free(p);
```

4. Why Doubly Linked-List Deletion Is Different

In a singly linked list, each node only knows the next node:

```
[data | next]
```

So if we want to delete a middle node, we usually need two pointers:

```
q = previous node  
p = node to delete
```

Example singly linked-list deletion:

```
10 -> 20 -> 30 -> 40 -> NULL
   q     p
```

To delete `30`, we need `q` so we can write:

```
q->next = p->next;
```

But in a doubly linked list, each node already has a pointer to its previous node:

```
[prev | data | next]
```

So if `p` points to the node we want to delete, we can use:

```
p->prev
p->next
```

That means the node itself gives us access to both neighbors.

This is the main advantage of a doubly linked list during deletion.

5. Deletion Index Convention

Module 20 uses **1-based indexing** for deletion.

Index	Meaning
1	Delete the first node
2	Delete the second node
3	Delete the third node
n	Delete the last node

Example list:

```
10 <-> 20 <-> 30 <-> 40
```

This list has 4 nodes.

Valid deletion indexes are:

1, 2, 3, 4

Invalid indexes are:

0, -1, 5, 100

Important:

For deletion, index 0 is invalid.

This is different from insertion.

In insertion, index 0 usually means insert before the first node.

In deletion, index 1 means delete the first node.

6. Why Index Validation Is Needed

Before deleting, we must check whether the index is valid.

If the index is invalid, the function should not change the list.

Validation rule:

```
if (index < 1 || index > length(p)) {  
    return -1;  
}
```

Meaning:

Condition	Meaning
<code>index < 1</code>	The index is too small. Deletion starts from index 1.
<code>index > length(p)</code>	The index is larger than the number of nodes.

If the index is invalid, return -1.

Returning -1 means:

Deletion failed.

7. Helper Function: Length

To validate the index, we need to know how many nodes are in the list.

So we use a `length()` function.

```
int length(struct Node *p) {  
    int len = 0;  
  
    while (p != NULL) {  
        len++;  
        p = p->next;  
    }  
  
    return len;  
}
```

Explanation

```
int len = 0;
```

This starts the count at zero.

```
while (p != NULL)
```

This loop continues while `p` points to a real node.

```
len++;
```

This counts the current node.

```
p = p->next;
```

This moves to the next node.

```
return len;
```

This returns the final number of nodes.

Why do we stop at `NULL` ?

Because this is a **linear doubly linked list**.

A linear doubly linked list has a clear end:

```
last->next = NULL
```

So traversal stops when `p == NULL`.

8. Main Deletion Cases

Doubly linked-list deletion has three important cases:

1. Delete the first node
2. Delete a middle node
3. Delete the last node

The first node is special because deleting it changes the external pointer `first`.

Middle and last deletion both use the same main idea:

```
Connect the node before p to the node after p.
```

Case 1: Delete the First Node

9. Meaning

Deleting the first node means removing the node pointed to by `first`.

Example:

```
first  
|
```

```
v
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Delete index **1**.

The node containing **10** must be removed.

Final list:

```
first
|
v
NULL <- 20 <-> 30 -> NULL
```

10. Steps to Delete the First Node

Step 1: Save the first node in **p**

```
p = first;
```

Now **p** points to the node that will be deleted.

```
p
|
v
10 <-> 20 <-> 30
^
|
first
```

We need **p** because after moving **first**, we still need a pointer to the old first node so we can free it.

Step 2: Save the deleted value

```
x = p->data;
```

If the deleted node contains **10**, then:


```
x = 10
```

We save the value before freeing the node.

After freeing a node, we should not access its data.

Step 3: Move `first` to the second node

```
first = first->next;
```

Now `first` points to `20`.

```
p
|
v
10 <-> 20 <-> 30
      ^
      |
    first
```

Step 4: Make the new first node's `prev` equal to `NULL`

```
if (first != NULL) {
    first->prev = NULL;
}
```

Why?

Because the first node of a doubly linked list must not have a previous node.

So after deleting `10`, node `20` becomes the first node.

Therefore:

```
20->prev = NULL;
```

Step 5: Free the old first node

```
free(p);
```

This releases the memory used by the deleted node.

Step 6: Return the deleted value

```
return x;
```

If node `10` was deleted, return:

```
10
```

11. Head Deletion Code Pattern

```
p = first;
x = p->data;
first = first->next;

if (first != NULL) {
    first->prev = NULL;
}

free(p);
return x;
```

12. Why `if (first != NULL)` Is Needed

Suppose the list has only one node:

```
first
|
v
NULL <- 10 -> NULL
```

Now delete index `1`.

After this line:

```
first = first->next;
```

`first` becomes `NULL`.

So this would be unsafe:

```
first->prev = NULL;
```

Why?

Because `first` is not pointing to a real node anymore.

It is `NULL`.

So we must write:

```
if (first != NULL) {  
    first->prev = NULL;  
}
```

This handles both cases:

Situation	What Happens
List had many nodes	The new first node's <code>prev</code> becomes <code>NULL</code> .
List had one node	<code>first</code> becomes <code>NULL</code> , so we skip <code>first->prev</code> .

Case 2: Delete a Middle Node

13. Meaning

A middle node has both a previous node and a next node.

Example:

```
NULL <- 10 <-> 20 <-> 30 <-> 40 -> NULL
```

Delete 30.

Here:

```
20 is before 30.  
40 is after 30.
```

Before deletion:

```
20 <-> 30 <-> 40  
      ^  
      p
```

After deletion:

```
20 <-> 40
```

14. Required Link Changes

If p points to 30, then:

```
p->prev points to 20  
p->next points to 40
```

To remove 30, we need:

```
20->next = 40;  
40->prev = 20;
```

Using p, this becomes:

```
p->prev->next = p->next;  
p->next->prev = p->prev;
```

15. Dry Run: Delete Middle Node

Original list:

```
NULL <- 10 <-> 20 <-> 30 <-> 40 -> NULL
```

Delete index .

Index means delete the third node, which is .

Move to the third node:

```
NULL <- 10 <-> 20 <-> 30 <-> 40 -> NULL
                        ^
                        p
```

Now do:

```
p->prev->next = p->next;
```

Meaning:

```
20->next = 40
```

Then do:

```
p->next->prev = p->prev;
```

Meaning:

```
40->prev = 20
```

Then:

```
free(p);
```

Final list:

```
NULL <- 10 <-> 20 <-> 40 -> NULL
```

Case 3: Delete the Last Node

16. Meaning

The last node has no next node.

Example:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

The last node is `30`.

For the last node:

```
p->next == NULL
```

If we delete `30`, the final list should be:

```
NULL <- 10 <-> 20 -> NULL
```

Node `20` becomes the new last node.

So:

```
20->next = NULL;
```

17. Why Last-Node Deletion Needs a Safety Check

Suppose `p` points to the last node:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
                        ^
                        p
```

This line is correct:

```
p->prev->next = p->next;
```

Since `p->next` is `NULL`, this becomes:

```
20->next = NULL;
```

That is exactly what we want.

But this line is unsafe:

```
p->next->prev = p->prev;
```

Why?

Because `p->next` is `NULL`.

So this would mean:

```
NULL->prev
```

That is invalid.

Correct version:

```
if (p->next != NULL) {
    p->next->prev = p->prev;
}
```

This check means:

```
Only update the next node's prev if a next node actually exists.
```

18. Last-Node Deletion Code Pattern

```
p->prev->next = p->next;

if (p->next != NULL) {
    p->next->prev = p->prev;
}

free(p);
```

For the last node, `p->next` is `NULL`, so the `if` block is skipped.

19. Full C Code: Delete from a Doubly Linked List

```
int Delete(struct Node *p, int index) {
    int x;

    if (index < 1 || index > length(p)) {
        return -1;
    }

    // Case 1: Delete the first node
    if (index == 1) {
        p = first;
        x = p->data;

        first = first->next;

        if (first != NULL) {
            first->prev = NULL;
        }

        free(p);
        return x;
    }

    // Case 2: Delete any node after the first node
    for (int i = 0; i < index - 1; i++) {
        p = p->next;
    }

    x = p->data;
```



```
p->prev->next = p->next;

if (p->next != NULL) {
    p->next->prev = p->prev;
}

free(p);
return x;
}
```

20. Explanation of the Delete Function

Function header

```
int Delete(struct Node *p, int index)
```

Meaning:

Part	Meaning
int	The function returns the deleted value.
Delete	Function name.
struct Node *p	Pointer used to move through the list.
int index	Position of the node to delete.

Invalid index check

```
if (index < 1 || index > length(p)) {
    return -1;
}
```

This protects the list from invalid deletion.

If the index is invalid, the list is not changed.

Delete first node

```
if (index == 1) {
```

This handles the special first-node case.

Why special?

Because deleting the first node changes `first`.

Move to the target node

```
for (int i = 0; i < index - 1; i++) {  
    p = p->next;  
}
```

This moves `p` to the node that should be deleted.

Example:

If `index = 3`, then `p` must move two times:

```
Start at 1st node.  
Move once to 2nd node.  
Move twice to 3rd node.
```

Save deleted data

```
x = p->data;
```

Save the data before freeing the node.

Connect previous node to next node

```
p->prev->next = p->next;
```

This makes the previous node skip the deleted node.

Connect next node back to previous node

```
if (p->next != NULL) {  
    p->next->prev = p->prev;  
}
```

This fixes the backward link.

The `if` check protects us when deleting the last node.

Free the deleted node

```
free(p);
```

This releases heap memory.

Return deleted value

```
return x;
```

This returns the value that was removed.

21. Full Dry Run: Delete Index 3

Given list:

```
NULL <- 10 <-> 20 <-> 30 <-> 40 <-> 50 -> NULL
```

Call:

```
Delete(first, 3);
```

Index `3` means delete `30`.

Step 1: Validate index

List length is 5.

Index 3 is valid because:

```
1 <= 3 <= 5
```

Step 2: Move p to the third node

Step	p points to
Start	10
Move 1	20
Move 2	30

Now:

```
NULL <- 10 <-> 20 <-> 30 <-> 40 <-> 50 -> NULL
                        ^
                        p
```

Step 3: Save deleted value

```
x = p->data;
```

So:

```
x = 30
```

Step 4: Fix forward link

```
p->prev->next = p->next;
```

Meaning:

```
20->next = 40
```

Step 5: Fix backward link

```
p->next->prev = p->prev;
```

Meaning:

```
40->prev = 20
```

Step 6: Free node 30

```
free(p);
```

Final result

```
NULL <- 10 <-> 20 <-> 40 <-> 50 -> NULL
```

Returned value:

```
30
```

Part B: Given-Pointer Deletion Trick

22. What Is Given-Pointer Deletion?

Sometimes, instead of deleting by index, we may already have a pointer to the node we want to delete.

Example:

```
NULL <- 10 <-> 20 <-> 30 <-> 40 -> NULL
                        ^
                        p
```

Here, `p` already points to node `30`.

So we do not need to search for index `3`.

23. Why This Is Easier in a Doubly Linked List

In a singly linked list, if we only have pointer `p`, we do not directly know the node before `p`.

So we may need to start from `first` and search for the previous node.

But in a doubly linked list:

```
p->prev
```

already points to the previous node.

So if `p` points to a middle or last node, deletion can be done directly.

24. Core Given-Pointer Deletion Logic

```
p->prev->next = p->next;

if (p->next != NULL) {
    p->next->prev = p->prev;
}

free(p);
```

Meaning:

1. Make the previous node point to the node after `p`.
 2. If the next node exists, make it point back to the previous node.
 3. Free `p`.
-

25. Time Complexity Advantage

If the pointer to the target node is already given:

Deletion can be $O(1)$.

Why?

Because we do not need to traverse from the beginning.

We already have:

```
p->prev  
p->next
```

But if deleting by index, we still need traversal to reach the target node.

So deletion by index is usually:

$O(n)$

Deletion by given pointer can be:

$O(1)$

Part C: Reversing a Doubly Linked List

26. What Reversal Means

Reversal means changing the direction of the whole list.

Original list:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

After reversal:

```
NULL <- 30 <-> 20 <-> 10 -> NULL
```

The last node becomes the first node.

The first node becomes the last node.

27. Reverse Display vs Real Reversal

Reverse display only prints the list backward.

It does not change the list.

Real reversal changes the links inside the nodes.

Example:

```
Original actual list:  
10 <-> 20 <-> 30
```

Reverse display output:

```
30 20 10
```

But the actual list is still:

```
10 <-> 20 <-> 30
```

Module 20 reversal is real reversal.

After real reversal, the actual list becomes:

```
30 <-> 20 <-> 10
```

28. Main Idea of Doubly Linked-List Reversal

Each node has two links:


```
prev and next
```

To reverse the list, we swap these two links in every node.

For each node:

```
temp = p->next;  
p->next = p->prev;  
p->prev = temp;
```

Meaning:

```
old next becomes new prev  
old prev becomes new next
```

29. Why Swapping `prev` and `next` Works

Consider one node:

```
10 <-> 20 <-> 30  
      ^  
      p
```

For node `20`:

```
20->prev = 10  
20->next = 30
```

After reversal, `20` should point like this:

```
20->prev = 30  
20->next = 10
```

So the old `next` becomes the new `prev`.

And the old `prev` becomes the new `next`.

That is why we swap the two links.

30. Why We Need a Temporary Pointer

If we do this directly:

```
p->next = p->prev;  
p->prev = p->next;
```

This is wrong.

Why?

Because after the first line, the original `p->next` is lost.

So we first save it:

```
temp = p->next;
```

Then we can safely swap:

```
p->next = p->prev;  
p->prev = temp;
```

The temporary pointer protects the old `next` address.

31. Why We Move Using `p = p->prev` After Swapping

This is one of the most important ideas in Module 20.

Before swapping:

```
p->next means the next node in the original list.
```

After swapping:

```
p->prev contains the original next node.
```

So after swapping one node, if we want to continue moving through the original list, we must use:

```
p = p->prev;
```

Not:

```
p = p->next;
```

Because `p->next` now points backward after the swap.

Beginner rule:

During doubly linked-list reversal, after swapping links, move using `p = p->prev`.

32. Full C Code: Reverse a Doubly Linked List

```
void Reverse(struct Node *p) {
    struct Node *temp;

    while (p != NULL) {
        temp = p->next;
        p->next = p->prev;
        p->prev = temp;

        // After swapping, p->prev is the original next node
        p = p->prev;

        if (p != NULL && p->next == NULL) {
            first = p;
        }
    }
}
```

33. Explanation of the Reverse Function

Function header

```
void Reverse(struct Node *p)
```

Meaning:

Part	Meaning
<code>void</code>	The function does not return a value.
<code>Reverse</code>	Function name.
<code>struct Node *p</code>	Pointer used to move through the list.

Create temporary pointer

```
struct Node *temp;
```

`temp` is used to help swap `prev` and `next` safely.

Loop through the list

```
while (p != NULL)
```

This continues until all nodes are processed.

Save original next

```
temp = p->next;
```

This saves the original next node before changing links.

Swap links

```
p->next = p->prev;  
p->prev = temp;
```

This reverses the direction of the current node.

Move to original next node

```
p = p->prev;
```

After swapping, `p->prev` contains the original next node.

So this moves forward through the original list.

Update `first`

```
if (p != NULL && p->next == NULL) {  
    first = p;  
}
```

This detects the new first node.

After reversal, the old last node becomes the new first node.

The new first node has:

```
p->next == NULL
```

So we update:

```
first = p;
```

34. Dry Run: Reverse `10 <-> 20 <-> 30`

Original list:

```
first  
|  
v  
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Call:

```
Reverse(first);
```

Step 1: **p** points to 10

Before swap:

```
10->prev = NULL  
10->next = 20
```

Save original next:

```
temp = p->next;
```

So:

```
temp = 20
```

Swap:

```
p->next = p->prev;  
p->prev = temp;
```

After swap:

```
10->prev = 20  
10->next = NULL
```

Move:

```
p = p->prev;
```

Since `p->prev` is `20`, `p` moves to `20`.

Step 2: p points to 20

Before swap:

```
20->prev = 10  
20->next = 30
```

Save original next:

```
temp = 30
```

After swap:

```
20->prev = 30  
20->next = 10
```

Move:

```
p = p->prev;
```

Now p moves to 30 .

Step 3: p points to 30

Before swap:

```
30->prev = 20  
30->next = NULL
```

Save original next:

```
temp = NULL
```

After swap:

```
30->prev = NULL
30->next = 20
```

Now **30** has no previous node.

So **30** is the new first node.

Final list:

```
first
|
v
NULL <- 30 <-> 20 <-> 10 -> NULL
```

35. Reversal Table Trace

Original list:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Step	p points to	Before swap	After swap	Move using
1	10	prev=NULL , next=20	prev=20 , next=NULL	p = p->prev goes to 20
2	20	prev=10 , next=30	prev=30 , next=10	p = p->prev goes to 30
3	30	prev=20 , next=NULL	prev=NULL , next=20	p = p->prev goes to NULL

Final list:

```
NULL <- 30 <-> 20 <-> 10 -> NULL
```


Part D: Complexity Analysis

36. Deletion Complexity

Delete first node

Time: $O(1)$
Extra space: $O(1)$

Why?

We do not need traversal.

We only update a fixed number of pointers.

Delete middle node by index

Time: $O(n)$
Extra space: $O(1)$

Why?

We may need to move through the list to reach the target node.

The pointer changes themselves are $O(1)$, but finding the node takes $O(n)$.

Delete last node by index

Time: $O(n)$
Extra space: $O(1)$

Why?

We may need to traverse from the first node to the last node.

Delete a given node pointer

Time: $O(1)$
Extra space: $O(1)$

Why?

If `p` already points to the node to delete, then `p->prev` gives direct access to the previous node.

No traversal is needed.

37. Reversal Complexity

Time: $O(n)$
Extra space: $O(1)$

Why time is $O(n)$:

Every node must be visited once.

Why space is $O(1)$:

Only a temporary pointer is used.

No extra array or recursion stack is needed.

38. Complexity Summary Table

Operation	Time	Extra Space	Reason
Delete first node	$O(1)$	$O(1)$	No traversal needed
Delete middle by index	$O(n)$	$O(1)$	Need to reach target node
Delete last by index	$O(n)$	$O(1)$	Need traversal to last node
Delete given node pointer	$O(1)$	$O(1)$	<code>prev</code> gives direct previous-node access
Reverse doubly linked list	$O(n)$	$O(1)$	Visit each node once and swap links

Part E: Common Beginner Mistakes

39. Mistake 1: Updating Only the Forward Link

Wrong:

```
p->prev->next = p->next;
```

Why wrong?

This only fixes the `next` direction.

The backward direction may still point to the deleted node.

Correct:

```
p->prev->next = p->next;

if (p->next != NULL) {
    p->next->prev = p->prev;
}
```

40. Mistake 2: Accessing `p->next->prev` When `p->next` Is `NULL`

Wrong:

```
p->next->prev = p->prev;
```

This crashes or causes undefined behavior if `p` is the last node.

Correct:

```
if (p->next != NULL) {
    p->next->prev = p->prev;
}
```

41. Mistake 3: Forgetting to Update `first` When Deleting the Head

Wrong:

```
free(first);
```

This loses the rest of the list.

Correct:

```
p = first;  
first = first->next;  
free(p);
```

But also remember:

```
if (first != NULL) {  
    first->prev = NULL;  
}
```

42. Mistake 4: Forgetting `first->prev = NULL`

After deleting the first node, the new first node must have:

```
first->prev = NULL;
```

Otherwise, it may still point backward to the deleted node.

That would make the list invalid.

43. Mistake 5: Moving with `p = p->next` During Reversal

Wrong during reversal:

```
p = p->next;
```

Why wrong?

Because after swapping, `p->next` no longer means the original next node.

Correct:

```
p = p->prev;
```

After swapping, `p->prev` contains the original next node.

44. Mistake 6: Forgetting to Update `first` After Reversal

After reversal, the old last node becomes the new first node.

If `first` is not updated, the program may still think the old first node is the beginning.

Correct idea:

```
Update first to point to the old last node.
```

Part F: Complete Mini Program Structure

This is a typical structure for testing Module 20 operations.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    struct Node *prev;
    int data;
    struct Node *next;
};

struct Node *first = NULL;

void create(int A[], int n) {
    struct Node *t;
    struct Node *last;

    if (n <= 0) {
```

```

        first = NULL;
        return;
    }

    first = (struct Node *)malloc(sizeof(struct Node));
    first->prev = NULL;
    first->data = A[0];
    first->next = NULL;
    last = first;

    for (int i = 1; i < n; i++) {
        t = (struct Node *)malloc(sizeof(struct Node));
        t->prev = last;
        t->data = A[i];
        t->next = NULL;
        last->next = t;
        last = t;
    }
}

void display(struct Node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
    printf("\n");
}

int length(struct Node *p) {
    int len = 0;

    while (p != NULL) {
        len++;
        p = p->next;
    }

    return len;
}

int Delete(struct Node *p, int index) {
    int x;

    if (index < 1 || index > length(p)) {
        return -1;
    }

    if (index == 1) {
        p = first;
    }
}

```

```

        x = p->data;

        first = first->next;

        if (first != NULL) {
            first->prev = NULL;
        }

        free(p);
        return x;
    }

    for (int i = 0; i < index - 1; i++) {
        p = p->next;
    }

    x = p->data;

    p->prev->next = p->next;

    if (p->next != NULL) {
        p->next->prev = p->prev;
    }

    free(p);
    return x;
}

void Reverse(struct Node *p) {
    struct Node *temp;

    while (p != NULL) {
        temp = p->next;
        p->next = p->prev;
        p->prev = temp;

        p = p->prev;

        if (p != NULL && p->next == NULL) {
            first = p;
        }
    }
}

int main() {
    int A[] = {10, 20, 30, 40, 50};

    create(A, 5);

```

```

    printf("Original list: ");
    display(first);

    int deleted = Delete(first, 3);
    printf("Deleted value: %d\n", deleted);

    printf("After deletion: ");
    display(first);

    Reverse(first);

    printf("After reversal: ");
    display(first);

    return 0;
}

```

Expected output:

```

Original list: 10 20 30 40 50
Deleted value: 30
After deletion: 10 20 40 50
After reversal: 50 40 20 10

```

Part G: Key Patterns to Memorize

45. Delete First Node Pattern

```

p = first;
x = p->data;
first = first->next;

if (first != NULL) {
    first->prev = NULL;
}

free(p);
return x;

```

Use this when:


```
index == 1
```

46. Delete Middle or Last Node Pattern

```
p->prev->next = p->next;

if (p->next != NULL) {
    p->next->prev = p->prev;
}

free(p);
```

Use this when:

```
index > 1
```

47. Reverse Node Link Pattern

```
temp = p->next;
p->next = p->prev;
p->prev = temp;
p = p->prev;
```

Use this for every node during reversal.

Part H: Concept Checks

48. Concept Check 1

Question:

Why is deleting a node easier in a doubly linked list than in a singly linked list?

Answer:

Because a doubly linked-list node has `prev`, so it can directly access the node before it.

49. Concept Check 2

Question:

Why do we check `p->next != NULL` before writing `p->next->prev = p->prev`?

Answer:

Because if `p` is the last node, then `p->next` is `NULL`, and `NULL->prev` is invalid.

50. Concept Check 3

Question:

Why must we set `first->prev = NULL` after deleting the first node?

Answer:

Because the new first node should not point backward to any node.

51. Concept Check 4

Question:

In reversal, why do we move using `p = p->prev` after swapping?

Answer:

Because after swapping, `p->prev` contains the original next node.

52. Concept Check 5

Question:

What does reversal do to each node?

Answer:

It swaps the node's `prev` and `next` links.

Part I: Final Summary

Module 20 teaches two main operations on doubly linked lists.

Deletion Summary

Deletion removes one node from the list.

For the first node:

```
p = first;
first = first->next;

if (first != NULL) {
    first->prev = NULL;
}

free(p);
```

For a middle or last node:

```
p->prev->next = p->next;

if (p->next != NULL) {
    p->next->prev = p->prev;
}

free(p);
```

Main deletion rule:

```
Connect the deleted node's previous node to the deleted node's next node.
```

Reversal Summary

Reversal flips the whole list.

Original:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Reversed:

```
NULL <- 30 <-> 20 <-> 10 -> NULL
```

Core reversal pattern:

```
temp = p->next;  
p->next = p->prev;  
p->prev = temp;  
p = p->prev;
```

Main reversal rule:

```
For every node, swap prev and next.
```

53. One-Line Beginner Memory Rule

```
Deletion fixes the neighbors. Reversal swaps the directions.
```